

嵌入式C编码规范 — AI编程工具版

这是什么？

这是一套**嵌入式C语言工程化编码规范的AI Skill**（AI编码规范）。

简单来说：把这套规范导入你的AI编程工具后，AI在帮你写嵌入式C代码时，会**自动遵循这套工程化标准**，包括：

- SOLID原则在C语言中的实践
- 面向对象设计模式（状态机、观察者、策略、工厂等）
- Clean Code + 华为编码规范
- 内存安全与线程安全
- 硬件交互规则（HAL验证、非阻塞设计）
- 每次代码修改后的12阶段安全检查

不再需要你逐条提醒AI"要用const""要检查NULL""不要用魔法数字"——导入后它会自动做到。

文件说明

嵌入式C编码规范_AI编程工具版/

- ├ 使用说明.md ← 你正在看的这个文件
- ├ SKILL.md ← AI编码规范主文件（导入到AI工具中的文件）
- └ references/ ← 详细参考文档（SKILL.md 会引用这些文件）
 - ├ architecture.md ← 架构设计
 - ├ design-patterns.md ← 设计模式
 - ├ clean-code.md ← Clean Code与华为编码规范
 - ├ code-style.md ← 代码风格
 - ├ memory-safety.md ← 内存安全
 - ├ hardware-interaction.md ← 硬件交互
 - └ safety-checklist.md ← 安全检查清单

如何导入到 Claude Code

Claude Code 支持 Skill 功能，可以让 Claude 在编码时自动参考你的编码规范。

步骤

1. 将 `SKILL.md` 和整个 `references/` 目录复制到 Claude Code 的 skills 目录：

```
~/.claude/skills/embedded-c-coding/ ├── SKILL.md └── references/ ├──  
architecture.md ├── design-patterns.md ├── clean-code.md ├── code-style.md  
└── memory-safety.md ├── hardware-interaction.md └── safety-checklist.md
```

- macOS/Linux: `~/.claude/skills/embedded-c-coding/`
 - Windows: `C:\Users\你的用户名\.claude\skills\embedded-c-coding\`
2. 重启 Claude Code，Skill 会自动加载。
 3. 之后当你让 Claude 写嵌入式C代码时，它会自动按照这套规范执行。

效果

导入后，Claude Code 在以下场景会自动应用规范：

- 编写新的C模块/驱动
- 修改现有C代码
- Review代码
- 提到嵌入式、MCU、STM32、驱动、固件开发等关键词时

如何在 Cursor 中使用

Cursor 支持通过 `.cursorrules` 文件或项目说明来自定义AI行为。

方案一：作为 `.cursorrules` 文件

1. 将 `SKILL.md` 的内容复制到项目根目录下的 `.cursorrules` 文件中
2. 将 `references/` 目录复制到项目根目录下
3. Cursor 会自动读取 `.cursorrules` 并在编码时遵循

方案二：作为项目说明

1. 在 Cursor 中打开设置 → Project → Project Instructions

2. 将 `SKILL.md` 的内容粘贴进去
3. 将 `references/` 目录放到项目中

效果

导入后，Cursor 的AI助手在你的嵌入式项目中会：

- 自动使用封装模式、不透明指针
- 写出符合SOLID原则的模块结构
- 添加NULL检查、错误传播、const修饰
- 避免魔法数字，使用宏定义
- 驱动代码自动采用非阻塞+工作线程模式
- 代码修改后自动执行安全检查

导入后的效果对比

导入前（普通AI写的嵌入式代码）

```
int data;
int flag = 0;

void process() {
    data = read_sensor();
    if (data > 100) flag = 1;
    HAL_UART_Transmit(&huart1, &data, 1, 1000);
}
```

问题：全局变量、魔法数字、直接调用HAL、无错误处理、无封装。

导入后（按照规范写的代码）

```
#define SENSOR_THRESHOLD (100)
#define TX_TIMEOUT_MS    (1000U)

static int s_sensor_value;

int Sensor_Read(Sensor_t *self, int32_t *value)
{
    ASSERT(self != NULL);
```

```
ASSERT(value ≠ NULL);

int ret = self→ops→read(self→ctx, value);
if (ret ≠ ERR_OK) {
    return ret;
}

if (*value > SENSOR_THRESHOLD) {
    Subject_Notify(&self→subject,
                  EVT_THRESHOLD_EXCEEDED,
                  value);
}
return ERR_OK;
}
```

差距一目了然：封装、错误处理、const、宏常量、观察者模式通知。

常见问题

Q：这套规范适合什么项目？ A：适合所有嵌入式C项目，特别是使用RTOS（FreeRTOS等）的项目、需要高可靠性的安全关键系统、以及使用STM32等MCU的项目。

Q：我用的不是Claude/Cursor，能用吗？ A：可以。SKILL.md本质上是一份结构化的编码规范文档。任何支持自定义Prompt/系统提示词的AI工具都可以使用。将内容粘贴到工具的系统提示词或项目上下文中即可。

Q：规范内容可以修改吗？ A：当然可以。根据你的项目实际情况调整规则。比如如果你的项目不用QPC，可以去掉QPC相关部分。

获取更多

- **B站搜索「兆鸣嵌入式」** 观看完整教程（详细讲解+代码实战）
- 公众号搜索「兆鸣嵌入式」回复「交流」加技术交流群
- 抖音搜索「兆鸣嵌入式」看短视频精华版